

CS3210-MASTER-CHEAT-SHEET

CS 3210 — Master Cheat Sheet

All Lectures (02–15) · Spring 2026 · Francisco Romero · Georgia Tech

Focus: **Step-by-step flows**, **tradeoffs**, and **comparisons** — the exam-heavy material.



Table of Contents

1. [Boot Process](#)
 2. [What Happens When You Call a Syscall?](#)
 3. [What Happens on a Page Fault?](#)
 4. [What Happens on a Context Switch?](#)
 5. [What Happens on fork\(\)?](#)
 6. [Isolation Mechanisms](#)
 7. [Segmentation vs Paging](#)
 8. [Single-Level vs Multi-Level Page Tables](#)
 9. [TLB](#)
 10. [Virtual Memory Tricks](#)
 11. [Copy-on-Write Deep Dive](#)
 12. [Swapping & Page Replacement](#)
 13. [Kernel Architectures: Monolithic vs Micro vs Exo](#)
 14. [Threading Models: M:1 vs 1:1 vs M:N](#)
 15. [Interrupts: Types, Handling, IDT](#)
 16. [Concurrency: Races, Locks, Ordering](#)
 17. [Deadlock](#)
 18. [Scheduling Algorithms](#)
 19. [Condition Variables & Sleep/Wakeup](#)
 20. [x86 Registers & CPU Modes](#)
 21. [Quick Reference: PTE/PDE Flags](#)
-

1. Boot Process

The 3-Stage Chain

ROM/BIOS → Bootloader (bootasm.S + bootmain.c) → Kernel (entry.S → main.c)

Stage	Key Action	Code
BIOS	Hardware init, loads first 512B from disk	Firmware
Bootloader	Real→Protected mode, load ELF kernel, jump to entry	bootasm.S, bootmain.c
entry.S	Install entrypgdir (4MB superpage), enable paging, set stack	entry.S
main.c	kinit1, kvmalloc, start first process	main.c

Real Mode → Protected Mode

1. Load GDT (Global Descriptor Table) – sets up segment descriptors
2. Set CR0 bit 0 = 1 – enables protected mode
3. Far jump (ljmp) – flushes instruction pipeline, commits mode switch

Reset vector: x86 starts at `0xFFFFFFF0` — hardwired ROM address.

Boot Page Table Trick

```
// entrypgdir: two mappings to same physical memory
[0]                = PA[0..4MB]    // identity map during transition
[KERNBASE>>PDXSHIFT] = PA[0..4MB] // kernel runs at 0x80000000 (KERNBASE)
// PTE_PS = 4MB superpage (skips second page table level)
```

Why two mappings? During the transition in entry.S, the CPU is executing at a low physical address (e.g., 0x10000C). After turning on paging, if there was only the KERNBASE mapping, the next instruction fetch at the low address would fault. The `[0]` mapping keeps execution working until the code jumps to the high-address version.

2. What Happens When You Call a Syscall?

Full Step-by-Step Flow

```
write(1, buf, n)    ← user calls library wrapper
    ↓
usys.S: SYSCALL(write) macro
    → mov $SYS_write(16), %eax    ← put syscall number in eax
```

```

→ int $T_SYSCALL ($0x40)      ← fire software interrupt vector 64
↓
CPU hardware (automatically):
1. Look up IDT entry 64 (set up by tvinit)
2. Check CPL ≤ DPL in IDT entry (ensures user can invoke this)
3. Save SS, ESP, EFLAGS, CS, EIP onto KERNEL stack (not user stack!)
4. Load SS:ESP from TSS (switches to kernel stack)
5. Set CS:EIP from IDT entry 64 → jumps to alltraps in trapasm.S
↓
trapasm.S (alltraps):
    Push remaining registers → builds trap frame
    Call trap(tf)
↓
trap.c (trap()):
    if tf->trapno == T_SYSCALL:
        syscall()
↓
syscall.c (syscall()):
    num = proc->tf->eax      ← read syscall number from trap frame
    syscalls[num]()         ← dispatch via function table
↓
sysfile.c (sys_write()):
    Validate arguments (argint, argstr, argfd – USER POINTER CHECKING!)
    Do the actual write
    return result → goes into proc->tf->eax
↓
Return path: trapret in trapasm.S
    popal                  ← restore general purpose registers
    iret                   ← restores EIP, CS, EFLAGS, ESP, SS from
trap frame
↓
Back in user space after int instruction

```

Key Security Point: Sanitizing Arguments

The kernel **MUST** validate all user-provided pointers. A malicious process could pass a kernel address:

```

// sys_read(): safe argument fetching
int sys_write(void) {
    int fd, n;
    char *p;
    argint(0, &fd);    // fetch int from user stack (validated)

```

```

    argptr(1, &p, n); // fetch pointer (checks it's in user space!)
    argint(2, &n);
    // ... do the write
}

```

3. What Happens on a Page Fault?

Page fault = process accessed a virtual address with no valid PTE (P=0) or with wrong permissions (e.g., write to W=0 page).

Process accesses virtual address VA

↓

MMU: looks up PTE

↓

PTE has P=0 or permission violation

↓

CPU: raises trap vector 14 (T_PGFLT)

↓

trap.c: handles T_PGFLT

→ reads CR2 register to get the faulting VA

→ decides: is this a valid fault?

↓

Case 1: Lazy allocation fault

→ VA is in valid range but PTE missing → allocate frame, install PTE, return

Case 2: CoW fault

→ VA has W=0 but should be writable → copy frame, update PTE to W=1, return

Case 3: Swap fault

→ Page is on disk (P=0, but addr field has disk location) → read from disk, return

Case 4: Invalid access (segfault)

→ VA is not in any valid VMA → kill process (SIGSEGV)

4. What Happens on a Context Switch?

The Trigger

Either:

- **Timer interrupt** (vector 32): forces a context switch every ~10ms
- **Voluntary yield**: process calls `sleep()`, `exit()`, or `yield()`

Full Flow

Timer interrupt fires

↓

trap.c: calls `yield()`

↓

`yield()`:

`acquire(&ptable.lock)`

`proc->state = RUNNABLE`

`sched()`

↓

`sched()` → `swtch(&proc->context, cpu->scheduler)`

↓

switch in `swtch.S`:

 1. Read args from stack into `eax`, `edx` (before we change `esp`!)

 2. Push callee-saved regs: `edi`, `esi`, `ebx`, `ebp`

 3. Save current `esp` into `(*old)->esp` ← saves "where we are" in old thread

 4. Load `esp` from `new->esp`

← switches to scheduler's stack

 5. Pop: `ebp`, `ebx`, `esi`, `edi`

 6. `ret`

← returns to scheduler's saved

`eip`

↓

Now in `scheduler()` context (`proc.c`):

 // find next `RUNNABLE` process

 for each `proc` in `ptable`:

 if `proc->state == RUNNABLE`:

`switchvm(p)` ← load `p`'s page table into `CR3`

`p->state = RUNNING`

`swtch(&cpu->scheduler, p->context)` ← switch to `p`'s kernel

thread

↓

Back in `p`'s kernel thread

 Return through `sched()` → `yield()` → `trap()` → `trapret` → `iret`

↓

User space of new process

What swtch Saves vs trap

	swtch (voluntary)	trap handler (involuntary)
Triggered by	Explicit call	Hardware interrupt / exception
Saves	Callee-saved regs only (edi,esi,ebx,ebp,eip)	ALL registers (full trap frame)
Why?	Caller already saved its own regs per C convention	Interrupted code had no chance to save anything

5. What Happens on fork()?

Current xv6 fork() (Slow Copy)

fork() called:

1. allocproc(): find empty slot in ptable, allocate kernel stack
2. copyvm(): walk parent's entire page table
 - for each mapped page: kalloc() new frame + memmove() content
3. Copy trap frame (so child returns from fork() at same point as parent)
4. Copy file descriptors
5. Set child->parent = parent
6. Set child->state = RUNNABLE
7. return child PID to parent, return 0 to child

CoW fork() (Fast)

fork() called:

1. allocproc(): find empty slot in ptable, allocate kernel stack
2. cowvm(): walk parent's page table
 - for each mapped page: INCREMENT ref count
 - point child PTE to same physical frame
 - set W=0 on BOTH parent and child PTE
3. Copy trap frame, file descriptors, etc. (same)
4. Set child->state = RUNNABLE
5. return child PID to parent, return 0 to child

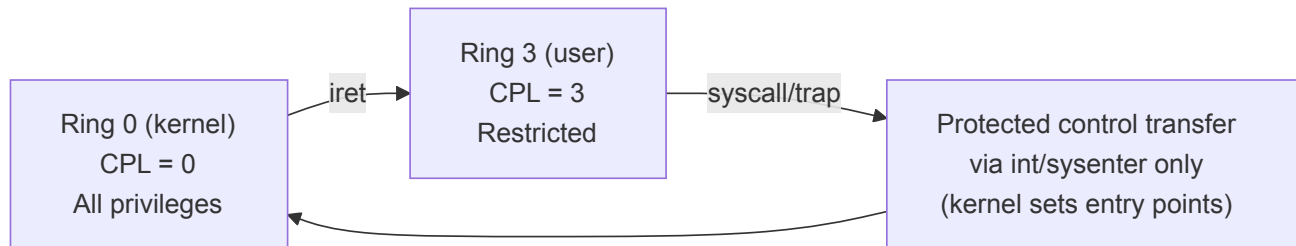
Later, when either process writes to a shared page:

- Page fault (W=0)

→ CoW handler: copy frame, update PTE W=1, decrement ref count on old frame

6. Isolation Mechanisms

Hardware Isolation in x86



Isolation Tradeoff

Dimension	Weaker Isolation	Stronger Isolation
Safety	Lower — failures propagate	Higher — failures contained
Usability	Higher — easy communication	Lower — need IPC
Performance	Higher — no overhead	Lower — context switch cost

The ideal: just enough isolation so failures don't propagate, while keeping communication efficient.

CPL (Current Privilege Level)

- Stored in bottom 2 bits of `%CS` register
- **Ring 0 (CPL=0):** kernel — full hardware access
- **Ring 3 (CPL=3):** user — restricted

Privileged operations at Ring 3 → immediate hardware exception.

Virtual Memory Isolation

Each process gets its own page directory (CR3). The MMU enforces:

- Process A cannot access Process B's physical frames (different PD)
- User code cannot access kernel pages (U bit in PTE = 0 for kernel pages)

CPU Isolation (Time-Slicing)

- Preemptive: timer interrupt forces yield every ~10ms

- Cooperative: process calls yield/sleep voluntarily
- Multiple processes share one CPU through rapid switching

7. Segmentation vs Paging

Side-by-Side

Feature	Segmentation	Paging
Unit	Variable-size segments	Fixed-size pages (4KB)
Physical memory layout	Contiguous blocks required	Any frames, anywhere
Fragmentation	External (gaps between segments)	Internal (wasted space within a page)
Permission granularity	Per segment (coarse)	Per page (fine)
V→P translation	One table lookup (fast)	Multi-level walk (slower)
Supports CoW/shared mem	Hard (contiguous constraint)	Easy (just remap PTEs)
Used today?	Mostly disabled on x86_64	Dominant

x86 Address Translation Pipeline

```

Logical Address (segment:offset)
    → [Segment Translation via GDT/LDT]
Linear Address
    → [Page Translation via PD → PT]
Physical Address

```

Modern OSes: all segments = base 0, limit 4GB → linear == virtual.

8. Single-Level vs Multi-Level Page Tables

Size Comparison

Level Count	Page Size	Address Space	Table Size	TLB miss cost
1 level	4MB	4GB (32-bit)	1 PD, 4KB	1 memory access

Level Count	Page Size	Address Space	Table Size	TLB miss cost
2 levels	4KB	4GB (32-bit)	PD + PTs (allocated on demand)	2 memory accesses
4 levels	4KB	256TB (64-bit)	PML4 + PDPT + PD + PT	4 memory accesses

Why Multi-Level Wins

Problem with flat table: A process with 4GB virtual space needs $2^{20} \times 4$ bytes = **4MB** just for the page table — per process. With 100 processes: 400MB wasted!

Multi-level solution: Only allocate page tables for regions that are actually used. A process using only 4KB of memory needs: 1 PD (4KB) + 1 PT (4KB) = **8KB** total.

The 32-bit 2-Level Walk

Virtual Address: [Dir: 10 bits][Table: 10 bits][Offset: 12 bits]

1. CR3 → physical address of PD
2. PD[Dir] → PDE → physical address of PT (or P=0 → page fault)
3. PT[Table] → PTE → physical page number (or P=0 → page fault)
4. Physical address = (PPN << 12) | Offset

Superpages (4MB in 32-bit x86)

Set PS=1 in PDE → skip the Page Table entirely. The PDE directly stores the 4MB-aligned physical address. TLB covers 4MB with one entry instead of 1024×4 KB entries.

Use cases: kernel bootstrap (entrypgdir), database buffer pools, GPU memory.

9. TLB

Translation Lookaside Buffer: hardware cache of recent VA→PA translations.

TLB vs Full Walk

	TLB Hit	TLB Miss
Frequency	~95%+	~5%
Cost	~1 cycle	~2–4 memory reads (one per page table level)
Action	Return cached PA	Full walk → store result in TLB

TLB Flush on Context Switch

When CR3 is loaded with a new process's page directory → entire TLB is flushed. This is a major hidden cost of context switching.

ASID (Address Space ID): Modern CPUs tag TLB entries with process ID. Allows TLB entries to survive context switches → no flush needed. x86 supports this via PCID (Process-Context Identifiers).

TLB Tradeoff

Small TLB	Large TLB
Less silicon area	More silicon area
More misses → more walks	Fewer misses
Better for simple workloads	Better for high-TLB-pressure workloads
Cheaper to build	More power/cost

10. Virtual Memory Tricks

Trick	Mechanism	Benefit	Cost
Copy-on-Write	W=0 on shared pages; copy on write trap	Fast fork, save memory	Fault on first write; ref counting complexity
Lazy Allocation	Don't create PTE until fault	Skip unnecessary zeroing/allocation	Fault latency on first access
Zero Page	All new pages → one read-only zero frame	Amortize zeroing cost	Fault on first write
Memory-Mapped Files	Map file into VA; demand-page from disk	Zero-copy I/O, easy random access	Consistency complexity, fault latency
Shared Memory	Multiple VAs → same PA	Fast IPC, shared libraries, CoW	Synchronization needed
Swapping	Evict pages to disk when RAM full	More logical memory than physical RAM	Huge latency if thrashing (~100μs vs ~100ns)
Page Pinning	mlock(): exempt from eviction	Real-time guarantees, DMA safety	Reduces evictable pool

11. Copy-on-Write Deep Dive

The Key Insight

Logical permission: Is this process allowed to write to this address? → YES

Physical permission: Should hardware trap on this write? → YES (W=0 in PTE)

These are independent! W=0 is a tripwire, not a true prohibition.

CoW State Machine

State 1: Page P is private (only one process uses it)
→ PTE: R=1, W=1 (normal writable)

State 2: Page P is shared (after fork, before any write)
→ Both processes' PTEs: R=1, W=0 (shared read-only)
→ Physical ref count = 2

State 3: Process A writes → page fault
→ Copy P to P' (new frame)
→ A's PTE now: points to P', R=1, W=1 (A has private copy)
→ B's PTE still: points to P, R=1, W=0 (B still shares, only reference)
→ Ref count of P drops to 1 → set P's W=1 (no need to copy on next write)

Reference Counting Rules

```
// On fork: for each shared page
ref_count[page]++

// On process exit / munmap: for each page in that mapping
ref_count[page]--
if (ref_count[page] == 0): free(page)

// On CoW write fault:
new_page = copy(old_page)
ref_count[old_page]--
if (ref_count[old_page] == 1): // only one owner left – un-protect it
    unprotect(old_page)        // set W=1 on remaining owner's PTE
```

```
proc_pte = new_page | W=1
ref_count[new_page] = 1
```

12. Swapping & Page Replacement

When to Evict

When a page fault occurs and all physical frames are in use, a page must be evicted to disk.

Replacement Algorithms

Algorithm	Strategy	Optimal?	Practical?
Belady's (OPT)	Evict page needed furthest in future	Yes (minimum faults)	No — needs future knowledge
LRU	Evict least recently used	Good approximation	Expensive — need per-access tracking
Clock	Approximate LRU using Accessed bit	Good	Yes — O(n) scan, no per-access overhead
FIFO	Evict oldest-loaded page	No	Simple but poor

Clock Algorithm Step-by-Step

Pages in memory as a circular list: P1(A=1) → P2(A=0) → P3(A=1) → P4(A=1) → (back to P1)

Hand starts at P1. Need to evict one page.

1. P1: A=1 → set A=0, advance. Hand: P1(A=0) → P2 → P3 → P4
2. P2: A=0 → EVICT P2! Replace with new page P5.

A=1 means "accessed recently, give it another chance." A=0 means "hasn't been used since last scan, evict it."

Dirty Page Optimization

If evicting a dirty page (D=1), must write to disk first (2× cost). Fix: add a second "write-back" clock hand that leads the eviction hand, pre-writing dirty pages to disk.

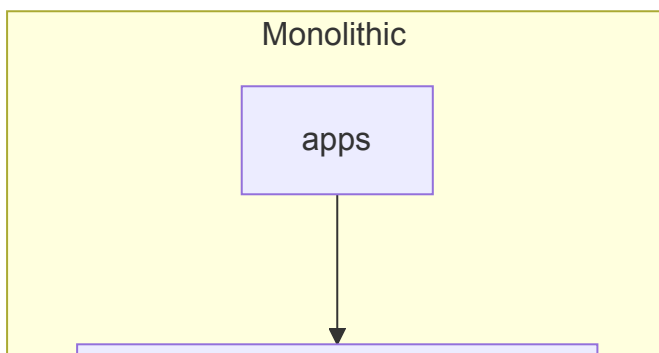
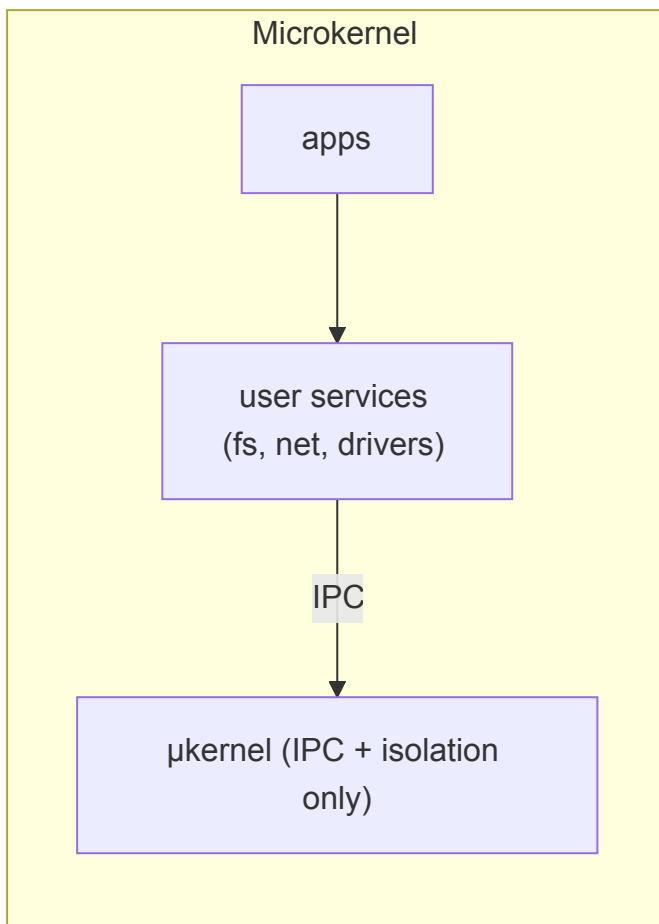
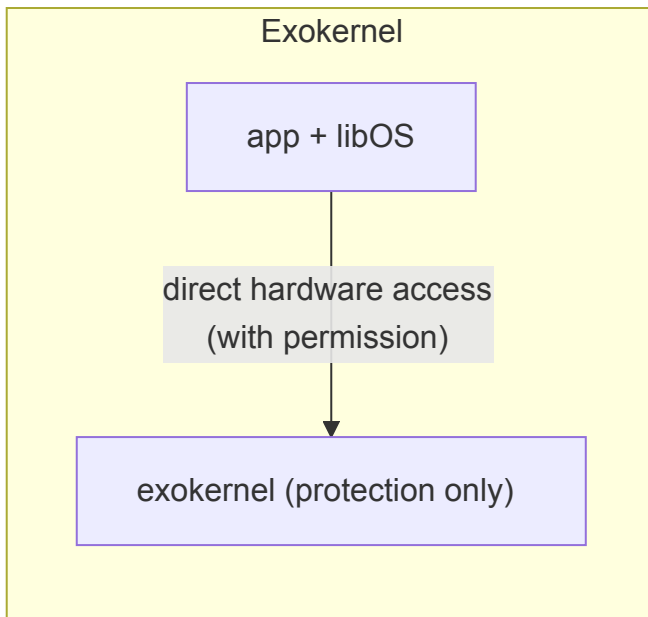
Belady's Anomaly

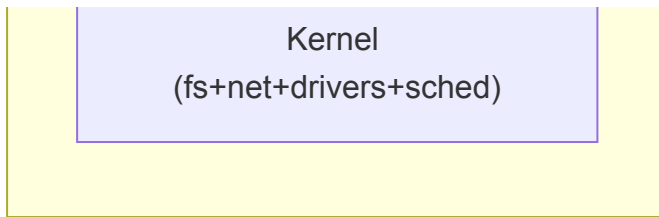
With FIFO (but NOT LRU or OPT): **adding more frames can cause MORE faults!** This is because FIFO doesn't consider recency of use.

13. Kernel Architectures: Monolithic vs Micro vs Exo

Side-by-Side

Feature	Monolithic	Microkernel	Exokernel
What's in kernel	Everything (fs, net, drivers)	Only IPC + minimal isolation	Only resource protection
User space	Applications only	All services (fs, drivers) + apps	Applications + libOS
Performance	⚡ High (direct function calls)	🐢 Lower (IPC for everything)	⚡ High (direct hardware)
Stability	Lower (bug = kernel crash)	Higher (service crash ≠ kernel crash)	Mixed
Extensibility	Hard (recompile kernel)	Easier (replace user-space service)	High (custom libOS)
Security	Lower (larger attack surface)	Higher (isolated services)	High (explicit capabilities)
Examples	Linux, macOS, xv6, Windows	Fuchsia, seL4, MINIX	MIT Exokernel





The Safety/Performance Tradeoff

Performance: Monolithic > Exokernel > Microkernel
 Safety: Microkernel > Exokernel > Monolithic
 Simplicity: Microkernel > Monolithic > Exokernel

Why Does Microkernel Have Lower Performance?

Every service call (e.g., read a file) requires crossing the user/kernel boundary multiple times:

- App → microkernel (IPC to filesystem service)
- Microkernel → filesystem service
- Filesystem service → microkernel (IPC reply)
- Microkernel → app

vs monolithic: App → kernel (one boundary crossing, direct function call inside kernel).

14. Threading Models: M:1 vs 1:1 vs M:N

Complete Comparison

	M:1 (User threads)	1:1 (Kernel threads)	M:N (Hybrid)
Creating threads	⚡ malloc + set up stack	🐢 kernel call (clone)	⚡ near M:1
Scheduling	User library decides	OS kernel decides	Both
Context switch	⚡ No kernel trap	🐢 Full trap required	🔄 Mixed
I/O blocking	💀 Blocks ALL threads	✅ Only blocks one	✅ Other threads continue
Parallelism (SMP)	❌ 1 CPU only	✅ All CPUs	✅ Up to N CPUs
Memory overhead	Low	High (kernel stack each)	Medium
Used by	Old green threads	Linux pthreads, xv6	Go, Erlang

Why Does 1:1 Context Switch Cost More?

M:1 switch:	M:N/1:1 switch:
1. save regs in user-space TCB	1. timer interrupt → TRAP to kernel
2. load new regs	2. save ALL regs to trap frame
3. jmp (no trap)	3. kernel scheduler runs
	4. swtch() saves kernel regs
	5. load new process kernel regs
	6. iret → restore new process regs
	(6 steps vs 3, involves kernel)

I/O Blocking: Why M:1 is Broken

Thread A calls read() on empty pipe:

M:1: kernel sees ONE kernel thread → puts it to sleep → ALL user threads stop!

1:1: kernel sees one kernel thread of many → puts it to sleep → OTHER kernel threads run

Linux clone() — Threads and Processes Unified

```
// Process (fork): own address space
clone(SIGCHLD) // minimal sharing

// Thread (pthread_create): shared address space
clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND | CLONE_THREAD,
stack)

// Container (Docker): isolated namespace
clone(CLONE_NEWPID | CLONE_NEWNET | CLONE_NEWNS | ...)
```

15. Interrupts: Types, Handling, IDT

Types of Traps (Broad Category)

Type	Cause	Example	Intentional?
System call	int 0x40 instruction	write(), read()	Yes (user → kernel)

Type	Cause	Example	Intentional?
Hardware interrupt	Device raises interrupt line	Keyboard press, timer tick	No (unpredictable)
Exception	CPU error condition	Page fault (vec 14), divide by zero (vec 0)	No

Interrupt Vector Table (IDT)

Vector 0: Divide by zero (exception)
 Vector 2: Non-Maskable Interrupt (NMI)
 Vector 13: General Protection Fault
 Vector 14: Page Fault (CR2 = faulting address)
 Vector 32: Timer (IRQ0 → clock interrupt → scheduling tick)
 Vector 33: Keyboard
 Vector 64: System Call (T_SYSCALL = 0x40 in xv6)

Maskable vs Non-Maskable

	Maskable (INTR)	Non-Maskable (NMI)
Can be disabled?	Yes — <code>cli</code> instruction (clear IF in EFLAGS)	No — always fires
Used for	Normal device I/O (keyboard, disk, timer)	Fatal errors (power failure, memory ECC error)
xv6 behavior	Disabled when holding spinlock	OS panics / dies

PIC / APIC / LAPIC

Old system (single CPU):
 [Devices] → [PIC (8259A)] → [CPU]

Modern multi-CPU:
 [Devices] → [IOAPIC] → [LAPIC per CPU] → [CPU]

LAPIC: local to each CPU — handles timer, IPI (inter-processor interrupts)
 IOAPIC: routes device interrupts to specific CPUs (affinity)

Interrupt Handling Flow

1. Device raises interrupt
2. PIC/APIC routes to CPU

3. CPU: finish current instruction, check EFLAGS.IF
4. Look up IDT[vector]:
 - Check $CPL \leq DPL$ in gate
 - Save SS, ESP, EFLAGS, CS, EIP to kernel stack (hardware)
 - Load SS:ESP from TSS (switch to kernel stack)
 - Load CS:EIP from IDT gate (jump to handler)
5. Handler runs, services device
6. Handler calls `iret` → restores all saved state
7. CPU resumes interrupted code

Interrupt vs Polling

	Polling	Interrupts
CPU usage	Burns 100% (busy wait)	Zero when idle
Latency	Very low (constant checking)	Slightly higher (save/restore overhead)
Ideal for	Ultra-high frequency events (10Gbps network)	Infrequent/unpredictable events (keyboard)

Surprising: for a 10Gbps network card, **polling beats interrupts** because traffic is so dense that the interrupt overhead dominates. DPDK (Data Plane Development Kit) uses this: dedicate a CPU core to polling the NIC.

16. Concurrency: Races, Locks, Ordering

Data Race Definition

Data race: two concurrent memory accesses to the same location, at least one is a write, and there's no ordering between them.

```
// Data race:
Thread 1: x = 1;           Thread 2: y = x; // reads x without lock
// No lock = no ordering guarantee = undefined behavior
```

Spinlock

```
// Acquire: spin until we successfully exchange 1 for 0
while (exchange(&lock, 1) != 0) { /* spin */ }
```

```
// Release: set lock back to 0
lock = 0;
```

Spinlocks in xv6: `acquire()` = disable interrupts + spin-acquire. `release()` = release + re-enable interrupts. **Why disable interrupts?** If we hold a spinlock and an interrupt fires that also tries to acquire the same lock → deadlock!

Total Store Ordering (TSO) — x86's Memory Model

```
What you write:    x = 1; r = y;
What CPU might do: r = y; x = 1;    (loads can be reordered before stores)
```

x86 guarantees:

- Write-Write ordering is preserved ✓
- Read-Write ordering is preserved ✓
- Write-Read ordering is **NOT** preserved ✗ (can be reordered!)

T1	T2
<code>movl \$1, (x)</code>	<code>movl \$1, (y)</code>
<code>movl (y), %eax</code>	<code>movl (x), %ebx</code>

Result: `%eax=0, %ebx=0` IS POSSIBLE (both loads reordered before both stores)

C11 Sequential Consistency Rule

"If you write data-race-free code, your code will behave sequentially consistently."

Use `<stdatomic.h>` atomic operations. Without them, you're at the mercy of TSO.

17. Deadlock

Deadlock: two or more threads/processes permanently blocked, each waiting for a resource held by the other.

Four Necessary Conditions (Coffman Conditions)

All four must be true for deadlock:

Condition	Meaning
Mutual exclusion	Resource can only be held by one at a time
Hold and wait	Holding resource(s) while waiting for another
No preemption	Resources can't be forcibly taken away
Circular wait	A waits for B, B waits for C, C waits for A

Prevention: break ANY one of these four conditions.

Dining Philosophers

3 philosophers, 3 chopsticks between them. Each needs 2. Each grabs left first, then right.

```
A grabs chopstick 0 (left), waits for 1
B grabs chopstick 1 (left), waits for 2
C grabs chopstick 2 (left), waits for 0
→ DEADLOCK (circular wait!)
```

Fix: impose a total ordering on resources. Number chopsticks 0, 1, 2. Always grab the lower-numbered one first. This breaks circular wait.

Deadlock vs Livelock vs Starvation

	What happens
Deadlock	Everyone is blocked, no progress
Livelock	Everyone is running but not making progress (e.g., two people keep stepping aside for each other)
Starvation	One thread is always deprioritized and never runs

18. Scheduling Algorithms

Metrics

Metric	What it measures
Throughput	Jobs completed per second
Turnaround time	Time from job arrival to completion
Response time	Time from request to first response

Metric	What it measures
Fairness	Equal CPU share to equal-priority processes
CPU utilization	% of time CPU is doing useful work

Algorithm Comparison

Algorithm	Strategy	Preemptive?	Good for	Bad for
FCFS	Run in arrival order	No	Simple batch jobs	Long jobs block short ones
Round Robin	Run each job for quantum q	Yes	Interactive (low response time)	High overhead if q too small
SRTF (Shortest Remaining Time)	Always run shortest remaining job	Yes	Minimizes avg turnaround	Starvation of long jobs
Priority	Run highest priority first	Yes	Real-time, differentiated	Starvation of low-priority
CFS (Linux)	Weighted fair share (vruntime)	Yes	General purpose	Complexity
EEVDF (new Linux)	Eligible Earliest Deadline First	Yes	Latency + fairness	Very complex

Round Robin: Quantum Size Tradeoff

Small quantum (e.g., 1ms)	Large quantum (e.g., 100ms)
Very responsive (great for interactive)	Less responsive
High overhead (lots of context switches)	Lower overhead
Poor throughput if many short jobs	Better throughput

Linux CFS (Completely Fair Scheduler)

Each process gets CPU time proportional to its weight. Tracks `vruntime` (virtual runtime): always schedule the process with smallest `vruntime` (most starved).

xv6 Scheduler

Simple round-robin: scan the process table, find first `RUNNABLE`, run it.

```
// proc.c (simplified)
void scheduler(void) {
    for(;;) {
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
            if(p->state != RUNNABLE) continue;
            proc = p;
            switchvm(p);    // load process's page table
            p->state = RUNNING;
            switch(&cpu->scheduler, proc->context);    // switch to process
        }
    }
}
```

19. Condition Variables & Sleep/Wakeup

CV Interface

```
wait(cv, lock):    // Atomically: release lock + put to sleep
                  // On wakeup: re-acquire lock
signal(cv):        // Wake ONE sleeping thread
broadcast(cv):     // Wake ALL sleeping threads
```

The Golden Rule: ALWAYS Use `while`

```
// ✓ CORRECT
while (!condition) cv.wait(lock);

// ✗ WRONG – race condition + spurious wakeups
if (!condition) cv.wait(lock);
```

Producer/Consumer Canonical Solution

```
// Producer
lock(mu);
q.push(item);
cv.signal();    // or broadcast()
unlock(mu);

// Consumer
```

```

lock(mu);
while (q.empty()) {    // ← WHILE not if!
    cv.wait(mu);        // atomically release mu + sleep
}                      // ← re-check on wakeup
item = q.pop();
unlock(mu);

```

Signal Placement

Signal can be placed before unlock (while holding lock) or after unlock. Both are correct. After-unlock is slightly more efficient (avoids a context switch that would immediately block on the lock again).

Spinlock vs Mutex vs CV

	Spinlock	Mutex (blocking lock)	Condition Variable
Wait behavior	Spin (burn CPU)	Sleep (yield CPU)	Sleep (yield CPU)
Use case	Very short critical sections ($< 1\mu\text{s}$)	Protect shared data	Wait for condition to become true
Overhead	Zero (no context switch)	Context switch overhead	Context switch overhead

20. x86 Registers & CPU Modes

Key Registers

Register	Purpose
EAX, EBX, ECX, EDX	General purpose (EAX = accumulator, return value, syscall number)
ESI, EDI	Source/Destination index (string ops)
EBP	Base pointer (stack frame)
ESP	Stack pointer
EIP	Instruction pointer (program counter)
EFLAGS	CPU status (ZF=zero, CF=carry, OF=overflow, IF=interrupt enable)
CR0	Control: bit 0 = protected mode enable, bit 31 = paging enable

Register	Purpose
CR2	Holds the faulting virtual address on page fault
CR3	Physical address of current page directory
CS	Code segment (lower 2 bits = CPL — current privilege level)

CPU Modes

Mode	Bits	Features
Real mode	16	Simple segmentation, max 1MB, no protection, boot mode
Protected mode	32	Full GDT, paging, privilege rings, 4GB address space
Long mode (x86_64)	64	4-level paging, 256TB address space

x86 Boot Transition

```

Power on → Real mode (16-bit, 0xFFFFFFFF0 reset vector)
    ↓ bootloder (bootasm.S)
Protected mode (32-bit, load GDT, set CR0.PE=1, far jump)
    ↓ entry.S
Paging enabled (load CR3, set CR0.PG=1)
    ↓ main.c
Full kernel

```

21. Quick Reference: PTE/PDE Flags

```

Bit 0: P (Present)      – Is page in RAM? P=0 → page fault
Bit 1: W (Writable)      – Can be written? W=0 → write fault (CoW
tripwire!)
Bit 2: U (User)         – User accessible? U=0 → kernel only
Bit 3: WT (Write-through)
Bit 4: CD (Cache disable)
Bit 5: A (Accessed)     – CPU sets on read. Used by clock algorithm
Bit 6: D (Dirty)        – CPU sets on write. Used for swap write-back
Bit 7: PS (Page Size)   – PDE only: 4MB superpage (skip PT level)
Bits 12–31: PPN (Physical Page Number)

```


Exam Hot Topics (Likely Questions)

Tracing a system call — be able to walk `write()` from user space to kernel and back, naming every file and step.

Page table walk — draw the 2-level hierarchy, trace a virtual address to physical.

Copy-on-write — explain the W-bit trick, what triggers a fault, what the handler does, why ref counting is needed.

Threading model comparison — why does M:1 I/O block all threads? Why is 1:1 context switching expensive? When is M:N better?

Monolithic vs microkernel — list tradeoffs (performance vs isolation vs stability), give real-world examples.

Deadlock — four conditions, dining philosophers solution, lock ordering rule.

Clock algorithm — step through an example, explain A bit and D bit roles.

TLB — why is there a flush on context switch, what does this cost, how does ASID help?

Interrupt vs exception vs syscall — distinguish the three, explain the IDT lookup flow.

Context switch cost — what gets saved/restored in swtch vs trap handler, and why the difference?

Memory model (TSO) — what can x86 reorder? What's the guarantee with data-race-free code?